

Competitive Programming

Section 38 Editorial: Static Queries and Binary Lifting

Competitive Programming Course

2026-07-10

Table of contents

1	Editorial overview	3
2	A. Static Range Minimum	4
2.1	Restatement	4
2.2	Algorithm	4
2.3	Correctness proof	4
2.4	Complexity	4
2.5	Full C++ solution	4
2.6	Full Python solution	5
3	B. Static Range GCD	7
3.1	Restatement	7
3.2	Algorithm	7
3.3	Correctness proof	7
3.4	Complexity	7
3.5	Full C++ solution	7
3.6	Full Python solution	8
4	C. Kth Ancestor	10
4.1	Restatement	10
4.2	Algorithm	10
4.3	Correctness proof	10
4.4	Complexity	10
4.5	Full C++ solution	10
4.6	Full Python solution	11
5	Testing discussion	13
6	Problem-specific upsolve notes	14
6.1	A. Static Range Minimum	14
6.2	B. Static Range GCD	14
6.3	C. Kth Ancestor	14
7	Stress-testing workflow	15

1 Editorial overview

Problem	Main idea
A. Static Range Minimum	See full algorithm and code below.
B. Static Range GCD	See full algorithm and code below.
C. Kth Ancestor	See full algorithm and code below.

2 A. Static Range Minimum

2.1 Restatement

Given an immutable array, answer minimum queries on subarrays.

2.2 Algorithm

Identify the operation that must be fast, store exactly the aggregate information needed for that operation, and update the affected logarithmic frontier. The lesson for this section explains the invariant in detail; the implementation below is a direct transcription of that invariant.

For this problem, every query is processed in the order given except offline queries explicitly marked as offline. The answer is emitted immediately when the operation asks for one.

2.3 Correctness proof

The maintained structure stores the exact summary promised by the lesson: prefix blocks for Fenwick trees, segment aggregates for segment trees, power-of-two jumps for static structures, or the active frontier for heap/Mo-style processing. Each update changes precisely the summaries whose represented set contains the changed element or interval. Each query decomposes the requested object into summaries that are disjoint, idempotently overlapping, or maintained as the current active window. The merge formula for those summaries is exactly the mathematical definition of the requested answer, so every printed value is correct. $\square$

2.4 Complexity

The optimized operations are logarithmic unless the lesson says the structure is static or offline. Memory is linear or $O(n \log n)$ for sparse-table/binary-lifting preprocessing.

2.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
```

```

if (!(cin >> n >> q)) return 0;
vector<int> lg(n + 1);
for (int i = 2; i <= n; i++) lg[i] = lg[i / 2] + 1;
int K = lg[n] + 1;
vector<vector<long long>> st(K, vector<long long>(n));
for (int i = 0; i < n; i++) cin >> st[0][i];
for (int k = 1; k < K; k++) {
    for (int i = 0; i + (1 << k) <= n; i++) {
        st[k][i] = min(st[k - 1][i], st[k - 1][i + (1 << (k - 1))]);
    }
}
while (q--) {
    int l, r;
    cin >> l >> r;
    --l; --r;
    int k = lg[r - l + 1];
    cout << min(st[k][l], st[k][r - (1 << k) + 1]) << '\n';
}
return 0;
}

```

2.6 Full Python solution

```

import sys, math

def main():
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    n, q = data[0], data[1]
    a = data[2:2 + n]
    log = [0] * (n + 1)
    for i in range(2, n + 1):
        log[i] = log[i // 2] + 1
    st = [a]
    k = 1
    while (1 << k) <= n:
        prev = st[-1]
        length = 1 << (k - 1)
        st.append([min(prev[i], prev[i + length]) for i in range(n - (1 << k) + 1)])
        k += 1
    idx = 2 + n
    out = []
    for _ in range(q):
        l, r = data[idx] - 1, data[idx + 1] - 1
        idx += 2
        k = log[r - l + 1]
        out.append(str(min(st[k][l], st[k][r - (1 << k) + 1])))

```

```
    print("\n".join(out))  
  
if __name__ == "__main__":  
    main()
```

3 B. Static Range GCD

3.1 Restatement

Given an immutable positive array, answer gcd queries on subarrays.

3.2 Algorithm

Identify the operation that must be fast, store exactly the aggregate information needed for that operation, and update the affected logarithmic frontier. The lesson for this section explains the invariant in detail; the implementation below is a direct transcription of that invariant.

For this problem, every query is processed in the order given except offline queries explicitly marked as offline. The answer is emitted immediately when the operation asks for one.

3.3 Correctness proof

The maintained structure stores the exact summary promised by the lesson: prefix blocks for Fenwick trees, segment aggregates for segment trees, power-of-two jumps for static structures, or the active frontier for heap/Mo-style processing. Each update changes precisely the summaries whose represented set contains the changed element or interval. Each query decomposes the requested object into summaries that are disjoint, idempotently overlapping, or maintained as the current active window. The merge formula for those summaries is exactly the mathematical definition of the requested answer, so every printed value is correct. \square

3.4 Complexity

The optimized operations are logarithmic unless the lesson says the structure is static or offline. Memory is linear or $O(n \log n)$ for sparse-table/binary-lifting preprocessing.

3.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
```

```

if (!(cin >> n >> q)) return 0;
vector<int> lg(n + 1);
for (int i = 2; i <= n; i++) lg[i] = lg[i / 2] + 1;
int K = lg[n] + 1;
vector<vector<long long>> st(K, vector<long long>(n));
for (int i = 0; i < n; i++) cin >> st[0][i];
for (int k = 1; k < K; k++) {
    for (int i = 0; i + (1 << k) <= n; i++) {
        st[k][i] = gcd(st[k - 1][i], st[k - 1][i + (1 << (k - 1))]);
    }
}
while (q--) {
    int l, r;
    cin >> l >> r;
    --l; --r;
    int k = lg[r - l + 1];
    cout << gcd(st[k][l], st[k][r - (1 << k) + 1]) << '\n';
}
return 0;
}

```

3.6 Full Python solution

```

import math
import sys

def main():
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    n, q = data[0], data[1]
    a = data[2:2 + n]
    log = [0] * (n + 1)
    for i in range(2, n + 1):
        log[i] = log[i // 2] + 1
    st = [a]
    k = 1
    while (1 << k) <= n:
        prev = st[-1]
        length = 1 << (k - 1)
        st.append([math.gcd(prev[i], prev[i + length]) for i in range(n - (1 << k) + 1)])
        k += 1
    idx = 2 + n
    out = []
    for _ in range(q):
        l, r = data[idx] - 1, data[idx + 1] - 1
        idx += 2
        k = log[r - l + 1]

```

```
        out.append(str(math.gcd(st[k][l], st[k][r - (1 << k) + 1])))
    print("\n".join(out))

if __name__ == "__main__":
    main()
```

4 C. Kth Ancestor

4.1 Restatement

In a rooted tree with root 1, answer queries asking for the k -th ancestor of a node, or -1 if it does not exist.

4.2 Algorithm

Identify the operation that must be fast, store exactly the aggregate information needed for that operation, and update the affected logarithmic frontier. The lesson for this section explains the invariant in detail; the implementation below is a direct transcription of that invariant.

For this problem, every query is processed in the order given except offline queries explicitly marked as offline. The answer is emitted immediately when the operation asks for one.

4.3 Correctness proof

The maintained structure stores the exact summary promised by the lesson: prefix blocks for Fenwick trees, segment aggregates for segment trees, power-of-two jumps for static structures, or the active frontier for heap/Mo-style processing. Each update changes precisely the summaries whose represented set contains the changed element or interval. Each query decomposes the requested object into summaries that are disjoint, idempotently overlapping, or maintained as the current active window. The merge formula for those summaries is exactly the mathematical definition of the requested answer, so every printed value is correct. $\square$

4.4 Complexity

The optimized operations are logarithmic unless the lesson says the structure is static or offline. Memory is linear or $O(n \log n)$ for sparse-table/binary-lifting preprocessing.

4.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
```

```

int n, q;
if (!(cin >> n >> q)) return 0;
int LOG = 1;
while ((1 << LOG) <= n + 5) LOG++;
vector<vector<int>> up(LOG, vector<int>(n + 1, 0));
for (int v = 2; v <= n; v++) cin >> up[0][v];
for (int k = 1; k < LOG; k++) {
    for (int v = 1; v <= n; v++) up[k][v] = up[k - 1][up[k - 1][v]];
}
while (q--) {
    int v;
    long long dist;
    cin >> v >> dist;
    for (int k = 0; k < LOG && v; k++) {
        if (dist & (1LL << k)) v = up[k][v];
    }
    cout << (v ? v : -1) << '\n';
}
return 0;
}

```

4.6 Full Python solution

```

import sys

def main():
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    n, q = data[0], data[1]
    LOG = max(1, (n + 5).bit_length())
    up = [[0] * (n + 1) for _ in range(LOG)]
    idx = 2
    for v in range(2, n + 1):
        up[0][v] = data[idx]
        idx += 1
    for k in range(1, LOG):
        for v in range(1, n + 1):
            up[k][v] = up[k - 1][up[k - 1][v]]
    out = []
    for _ in range(q):
        v, dist = data[idx], data[idx + 1]
        idx += 2
        bit = 0
        while dist and v:
            if dist & 1:
                v = up[bit][v] if bit < LOG else 0
            dist >>= 1
            bit += 1
    for v in out:
        print(v)

```

```
        out.append(str(v if v else -1))
    print("\n".join(out))

if __name__ == "__main__":
    main()
```

5 Testing discussion

The fixed samples cover visible behavior. The random tests generate many small operation sequences and compute expected outputs from the reference Python implementation, then the same expected files are used against both the C++ and Python submissions.

6 Problem-specific upsolve notes

6.1 A. Static Range Minimum

Constraint implication and trap. The array never changes, so pay $O(n \log n)$ once for $O(1)$ queries. Do not use this overlapping two-block query trick for sum: it is valid here only because min is idempotent.

For $[5, 2, 7, 1, 3, 4]$, query $[2, 5]$ has length 4 and reads one 2^2 block; query $[2, 6]$ uses blocks $[2, 5]$ and $[3, 6]$, whose overlap cannot alter a minimum. Pseudocode is: build $st[k][i] = \min(st[k-1][i], st[k-1][i + 2^{k-1}])$; for $[l, r]$, set $k = \lg[r-l+1]$ and take the min of the two length- 2^k blocks. Induction proves each table cell; the two blocks cover the query, proving the answer. Preprocessing/memory $O(n \log n)$, query $O(1)$. Test singleton and non-power-of-two ranges; convert input indices exactly once.

6.2 B. Static Range GCD

Use the identical table shape with gcd as merge. For $[12, 18, 6, 10, 15]$, query $[2, 5]$ has gcd 1. Gcd is associative and idempotent, so the same overlapping-cover proof applies. Complexity is again $O(n \log n)$ build and memory, $O(1)$ per query. C++ uses `std::gcd`; Python uses `math.gcd`. Test a length-one range, equal values, and a range whose answer becomes one early.

6.3 C. Kth Ancestor

Observation and trace. Let $up[k][v]$ be the exact 2^k -th ancestor, with zero meaning absent. Build by composing two half jumps. To lift 4 by three in the sample tree, take bit 0 to reach 2, then bit 1 to reach zero, so output -1; for distance zero output 4 unchanged.

```
build up[0][v]=parent[v]
up[k][v]=up[k-1][up[k-1][v]]
for each set bit k of distance: v=up[k][v]
answer -1 if v=0 else v
```

Induction on k proves the table's jump meaning; binary expansion uses exactly the requested number of parent edges, proving each query. Build/memory are $O(n \log n)$ and a query is $O(\log n)$. Choose LOG for the largest possible query distance, not just an arbitrary constant. Test root, zero distance, one-step, and a distance above depth.

7 Stress-testing workflow

Compare sparse-table answers to direct scans on small arrays. For ancestor queries, climb the parent array literally. Run `CP_TARGET=solution python3 sections/38_static_queries_binary_lifting/check.py`. Carry forward: overlap needs idempotence; binary lifting is the same power-of-two decomposition applied to functional-graph/tree jumps.